Remotion_Core.md

Módulo 1: Motor de Renderizado y Contexto Global

El núcleo arquitectónico de Remotion opera bajo un paradigma de evaluación determinista estricta, en el cual el estado de la interfaz de usuario se define como una función matemática pura del tiempo. A diferencia de las aplicaciones web convencionales, que reaccionan a eventos asíncronos y a la interacción del usuario en tiempo real, el motor de renderizado de Remotion aísla el árbol de componentes de React y lo vincula de manera inmutable a un número entero secuencial y absoluto denominado *frame*.[1] Este aislamiento garantiza que la salida de píxeles generada en el *frame* N sea algorítmicamente idéntica en cualquier entorno de ejecución, independientemente de la latencia de procesamiento o de las fluctuaciones en la red.[1]

Arquitectura del Tiempo y Sincronización Determinista

La arquitectura del tiempo en Remotion se estructura mediante la inyección de un contexto global de reloj de renderizado que se propaga a través de todo el árbol del Document Object Model (DOM) virtual. Durante el proceso de evaluación y exportación, el motor de Remotion instancia un entorno de navegador *headless* (usualmente implementado sobre Chromium mediante bibliotecas de control como Puppeteer) e incrementa el tiempo de manera artificial y secuencial.[1]

Por cada incremento en el bucle de renderizado, el motor actualiza el contexto del *frame* actual, fuerza la reconciliación síncrona del árbol de React y posteriormente extrae una captura de pantalla del DOM resultante.[1] La resolución del tiempo se rige por la siguiente relación escalar fundamental:

$$T_{segundos} = \frac{Frame_{absoluto}}{FPS}$$

Este modelo exige que toda la variabilidad visual, espacial y tipográfica dentro del componente sea impulsada exclusivamente por la mutación de este *frame* inyectado. El uso de variables globales no deterministas o funciones estocásticas no controladas (como Math.random()) corrompe la integridad del renderizado, dado que la orquestación asíncrona y distribuida del motor evaluaría resultados divergentes en diferentes hilos de procesamiento.[3] Para inyectar variabilidad matemática válida, la arquitectura provee generadores de números pseudoaleatorios basados en semillas criptográficas estáticas (random(seed)), asegurando así la replicabilidad temporal exacta de la topología del video.[3]

Variables Globales del Motor

El acceso al estado temporal absoluto y a la topología dimensional de la composición se expone mediante *hooks* de contexto estrictamente definidos. El motor bloquea la modificación

de la línea de tiempo a través de estos *hooks*, imponiendo un modelo de lectura determinista de un solo flujo.

Definición del Hook useCurrentFrame()

La función useCurrentFrame() constituye el vector principal de sincronización temporal. Este *hook* extrae el valor temporal actual del contexto inyectado por el motor y su retorno es de tipo number, representando la posición activa del cabezal de reproducción.[4]

El comportamiento topológico de este valor está sujeto a las siguientes reglas matemáticas y estructurales: El sistema opera basándose en un índice cero absoluto. El vector inicial de cualquier evaluación temporal es siempre 0.[4] Por consiguiente, el límite superior o el ciclo de vida máximo de una evaluación concluye de forma determinista en el *frame* correspondiente a la ecuación $durationInFrames - 1$.[5]

La resolución espacial del *frame* retornado se clasifica en relativa o absoluta dependiendo del encapsulamiento del componente en el árbol topológico. Cuando useCurrentFrame() se invoca en el nivel de la raíz de la composición, el motor proporciona el tiempo absoluto global. Sin embargo, si la invocación ocurre dentro del dominio de un nodo de desplazamiento temporal (como el componente estructural <Sequence>), el motor de contexto intercepta la propagación de la llamada y aplica una sustracción vectorial automática. El valor retornado pasa a ser un *frame* relativo al momento de inicio del contenedor envolvente, aislando matemáticamente al componente hijo de la línea de tiempo global.[4] Para escenarios que exigen una lectura del tiempo absoluto dentro de estructuras anidadas, la arquitectura requiere que el valor de la raíz se extraiga en el nivel superior y se propague explícitamente mediante inyección de propiedades a través del subárbol.[4]

Definición del Hook useVideoConfig()

El *hook* useVideoConfig() expone la topología estática y la configuración del entorno de la composición activa. Retorna un objeto inmutable de tipo VideoConfig cuya evaluación es constante a lo largo de todo el ciclo de vida del renderizado, previniendo mutaciones que corrompan el cálculo de los cuadros.[7]

La estructura determinista del objeto devuelto contiene los siguientes vectores paramétricos representados a través de una tabla de definición estricta:

Propiedad	Tipo	Comportamiento Arquitectónico y Topológico
width	number	Define la resolución horizontal absoluta del

		lienzo virtual en píxeles. El motor propaga las alteraciones dimensionales; si el *hook* es invocado por un componente encapsulado en una <Sequence> que inyecta su propio ancho, el valor retornado hereda y refleja dicha redefinición local.[7]
height	number	Establece la resolución vertical del lienzo en píxeles. Sigue la misma jerarquía de herencia contextual que el ancho, permitiendo el cálculo de matrices de aspecto y posicionamiento cartesiano relativo a contenedores modificados.[7]
fps	number	Frecuencia de muestreo maestro (Frames Per Second). Actúa como la constante universal para la conversión matemática entre el dominio del tiempo (t) y el dominio discreto de cuadros. Es imperativo para escalar cálculos físicos y de interpolación.[7]
durationInFrames	number	Cuantifica la longitud temporal del segmento activo. Si el invocador no posee un contexto envolvente, retorna el ciclo total de la raíz. En presencia de una topología de anidamiento (<Sequence>), el valor se reduce

		estrictamente a la duración de dicho encapsulamiento, permitiendo extrapolaciones dinámicas.[7]
id	string	Carga el identificador alfanumérico inmutable que vincula la composición de React con la instancia registrada en el servidor de compilación o en el proceso del CLI.[7]
defaultProps	object	Expone la matriz estática de valores JSON inyectada durante la instanciación arquitectónica de la composición.[7]
props	object	Contiene la carga útil final de propiedades inyectadas en el entorno tras resolver el árbol de fusiones paramétricas (superposición de *input props* sobre los predeterminados).[7]
defaultCodec	string	Define el algoritmo de compresión forzado para la orquestación del flujo de salida (ej. h264, vp8, prores), fundamental para condicionales de renderizado condicionado a transparencia.[7]

La extracción combinada de las variables de estos dos *hooks* permite la implementación de lógicas relativas. Por ejemplo, la opacidad vinculada a la fase de finalización de una secuencia puede modelarse sin constantes fijas evaluando la proporción $\frac{useCurrentFrame()}{durationInFrames}$, lo que asegura que las matemáticas de interpolación escalen automáticamente si se altera la topología del padre sin necesidad de refactorizar las curvas de animación internas.[3]

Módulo 2: Topología del Tiempo (Componentes Estructurales)

El ensamblaje de entidades temporales en Remotion exige una abstracción donde la jerarquía tridimensional del árbol de React mapea directamente una secuencia cronológica multidimensional. Los componentes topológicos no solo generan nodos en el DOM, sino que alteran la matriz dimensional del reloj inyectado, creando un control paramétrico sobre el flujo del tiempo.

Componente Raíz <Composition>

El nodo <Composition> representa la superestructura raíz y el punto de montaje principal del árbol lógico. A nivel de renderizado, este componente no inyecta elementos en el DOM directamente durante la reproducción, sino que actúa como un puente de memoria compartida, registrando metadatos y fronteras de contexto estricto en el núcleo de compilación de Remotion.[6]

El componente exige la configuración de parámetros primarios inmutables que dictan la resolución matricial de la evaluación subsecuente. Las propiedades de configuración determinista se estructuran de la siguiente manera:

Propiedad	Tipo / Estructura	Función Estructural y Reglas de Instanciación
id	string	Actúa como clave primaria para la invocación desde líneas de comando o el recolector SSR. El compilador impone una validación estricta permitiendo únicamente la presencia de caracteres alfanuméricos y el guion medio (-).[6]
component / lazyComponent	React.FC	Define el nodo de entrada del árbol a evaluar. El uso de lazyComponent integra los envoltorios de carga asíncrona de React Suspense en el ciclo de

		Webpack, optimizando drásticamente la latencia de arranque del entorno de desarrollo. Su implementación obliga a que el módulo apuntado posea un *export* predeterminado.[6]
fps	number	Inyecta el ritmo de pulso en hercios que escala el cálculo de iteración de useCurrentFrame() para todas las topologías subsecuentes.[8]
durationInFrames	number	Delimita el límite aritmético máximo del renderizador. Obliga al desmontaje del árbol raíz en la fase final.[8]
width / height	number	Fija el área del plano de proyecciones, limitando el volumen de cálculos matriciales de renderizado en los nodos inferiores.[8]
defaultProps	object	Inyecta una estructura estática como base para interpolaciones. El motor valida que los objetos contenidos sean estrictamente compatibles con la especificación JSON. Toda función o prototipo de clase compleja inyectada aquí resulta en un error fatal durante la compilación SSR, dado que la memoria compartida entre Node.js y Chromium exige serialización pura.[6]

calculateMetadata	function	Método asíncrono para mutar dimensionalidades o configuraciones antes de iniciar la evaluación de cuadros. Intercepta los *props* crudos permitiendo recalcular durationInFrames basándose en tiempos de respuesta de un video cargado o de una consulta en red previa a la inmovilización del árbol.[6]

Lógica Paramétrica de Desplazamiento: <Sequence>

El componente <Sequence> es el mecanismo operativo fundamental para aplicar transformaciones al vector del tiempo de manera local. Su arquitectura opera como un orden de orden superior, mutando simultáneamente las coordenadas temporales devueltas a los nodos hijos y la topología espacial subyacente en el DOM virtual.[6]

Las directivas paramétricas del desplazamiento son las siguientes:

- **Parámetro from**: Modifica el origen de la inyección temporal. Funciona como un vector de retraso si es un número positivo; todos los *frames* menores al escalar definido en from provocan que la capa de abstracción retorne null durante la reconciliación, evitando el montaje de recursos gráficos costosos prematuramente.[6] Un valor negativo efectúa un recorte espacial inverso: la línea de tiempo del descendiente se adelanta, causando que al momento de la renderización inicial del componente, su reloj lógico ya haya avanzado dicha magnitud de *frames*.[6]
- **Parámetro durationInFrames**: Inyecta una condición lógica que precipita el desmontaje prematuro del componente. Si la progresión cronológica relativa cruza el valor estipulado, el árbol es destruido automáticamente.[6]
- **Comportamiento de Diseño (layout)**: Este argumento manipula el flujo espacial. Si se omite, <Sequence> adopta el valor de "absolute-fill", lo cual obliga a Remotion a inyectar un envoltorio de nodo de capa ( ) con propiedades de cascada forzadas (position: absolute, top: 0, left: 0), apilando todo en el eje Z.[6] Cuando se establece en "none", el analizador destruye el envoltorio forzando a que la topología CSS dependa estrictamente del primer hijo DOM inyectado, invalidando paralelamente la transferencia de propiedades estéticas como className y style al nodo estructural.[6]
- **Pre-montaje (premountFor)**: La renderización determinista en un DOM sin cabeza a menudo causa picos en la redención del primer fotograma si este exige pintura intensiva o texturas WebGL. Este parámetro ordena la inyección en el árbol un número exacto de *frames* antes de que useCurrentFrame() devuelva cero, aplicando propiedades invisibles

(usualmente opacity: 0 encapsulada en la matriz styleWhilePremounted) para lograr decodificación anticipada.[6]

Matemática de Nodos en Cascada:

Al orquestar una jerarquía donde se anidan instancias de <Sequence>, el compilador topológico efectúa una sumatoria acumulativa. La latencia absoluta de aparición para un componente C ubicado a una profundidad estructural N a lo largo de una matriz continua se calcula mediante la fórmula de arrastre:

$$Frame_{relativo} = Frame_{absoluto} - \sum_{i=1}^{N} from_i$$

El desmontaje de componentes profundos se evalúa en una intersección de todos los límites impuestos por los nodos superiores, forzando cortes abruptos si el contenedor alcanza su cuota límite de tiempo independientemente del estado cronológico de sus descendientes.[6]

Orquestación de Ensamblaje Secuencial: <Series>

A diferencia de la complejidad aritmética requerida para sincronizar múltiples nodos <Sequence> en los que el usuario debe calcular estáticamente las compensaciones temporales de finalización para la transición del siguiente elemento, Remotion expone la capa de abstracción topológica <Series>. Su diseño arquitectónico suprime la responsabilidad algorítmica del desarrollo y calcula los vectores de montaje en tiempo de lectura.[9]

- **Restricciones de la Estructura Topológica**: La matriz paramétrica impone reglas severas: el elemento contenedor <Series> rechaza cualquier inyección de propiedad y su bucle interno evalúa de manera estricta que todos sus hijos inmediatos deriven de la instancia <Series.Sequence>. Esta uniformidad bloquea inyecciones anómalas que corromperían la medición del acumulador temporal.[9]
- **Lógica Funcional del Acumulador**: El entorno extrae la propiedad durationInFrames obligatoria de cada subsecuencia. Para determinar la coordenada estricta del marco de inicio del elemento posicional n, se aplica la siguiente aritmética de iteración automática:

$$Inicio_n = \sum_{i=0}^{n-1} (duration_i) + offset_n$$

- **Manipulación Fluida con el Parámetro offset**: El argumento offset admite valores numéricos bidireccionales en el hijo <Series.Sequence>. Una inyección positiva fuerza una pausa matemática, separando al nodo hijo del colapso final de su predecesor. Por su parte, la inclusión de un vector negativo desplaza la coordenada lógica de arranque de dicho nodo en dirección retrógrada. Esto permite solapamientos controlados y operaciones matemáticas donde dos elementos conviven cronológicamente durante las

transiciones de escena. Crucialmente en el contexto arquitectónico de Remotion, la aplicación de un offset negativo no colapsa el avance temporal de los nodos que le preceden en la lista estructurada; la alteración del tiempo es exclusivamente iterativa, mutando permanentemente los valores from calculados a partir de ese índice hacia los sucesores.[9]

Módulo 3: Fórmulas Matemáticas de Animación (Interpolación y Física)

Para construir movimiento determinista y alteraciones geométricas dentro del plano bidimensional sin dependencia en las imperfecciones de rendimiento del motor Chromium local, Remotion requiere funciones que abstraigan el transcurso del número entero de *frames* y lo traduzcan dinámicamente en valores de matriz escalar (grados de rotación, píxeles de traducción, niveles de canal alfa).

Interpolación Algebraica Lineal: La función interpolate()

La función interpolate() constituye el cálculo matemático fundamental de mapeo. Actúa traduciendo un vector dentro de un rango de referencia temporal hacia una proyección espacial o estética determinada. Su código fuente interno no requiere la manipulación del árbol en sí, permitiendo una síntesis matemática universal.[10]

La estructura sintáctica de ingreso demanda exactitud paramétrica:

interpolate(input, inputRange, outputRange, options)

- input: Representa el factor dependiente (I), usualmente alimentado por la constante cambiante extraída mediante useCurrentFrame() u otra ecuación controladora del tiempo.[10]
- inputRange: Define la topología escalar temporal exigiendo un sub-arreglo de coordenadas independientes. Este arreglo requiere monotonicidad estricta creciente (donde para un índice, $I_j < I_{j+1}$), sin lo cual la función falla al resolver el dominio algorítmico.[10]
- outputRange: Dicta la matriz objetivo del codominio. Debe poseer exactamente la misma longitud dimensional impuesta por inputRange.[10]

Mecanismo Matemático del Mapeo:

Dentro de su lógica algorítmica, el motor localiza el rango de segmento adyacente $[I_{min}, I_{max}]$ correspondiente a la posición de la variable I.

1. Efectúa una **normalización diferencial**, estableciendo el parámetro de base cero en función al dominio de progresión:

$$N = \frac{I - I_{min}}{I_{max} - I_{min}}$$

2. Evalúa una posible deformación si existe inyección de matriz de tensión matemática (Easing): $N' = f_{easing}(N)$.[10]
3. Desnormaliza el vector ajustándolo dentro de los bordes escalares definidos por la salida correspondiente al tramo actual del codominio $[O_{min}, O_{max}]$:

$$R = N' \times (O_{max} - O_{min}) + O_{min}$$

Tolerancia y Configuración de Extrapolación:

En escenarios donde I asume un valor matemático externo a las fronteras definidas por los anclajes de inputRange, las políticas de desbordamiento asumen un control estricto de acuerdo a la declaración del objeto de opciones (extrapolateLeft y extrapolateRight):

Parámetro Extrapolador	**Procedimiento Lógico Determinista**
extend (Por Defecto)	Permite que el núcleo matemático del motor continúe operando con la pendiente de progresión escalar de los últimos puntos contiguos válidos, rompiendo los límites de destino pero salvaguardando la continuidad geométrica.[10]
clamp	Impone una frontera estática. Obliga al retorno inmutable del valor final declarado dentro de outputRange para todo valor excesivo, estabilizando matrices como de canal de opacidad donde no existe coherencia estructural para proyecciones numéricas por encima de 1 o debajo de 0.[10]
wrap	Emula el retorno al origen, proyectando un ciclo continuo basado en la modulación del tramo. El motor calcula $\Delta = I_{max} -$ y somete la variable a la operación: $R = (((I - I_{min}) \bmod \Delta) + \Delta) \bmod$

	.[10]
identity	Destruye por completo la ecuación de correspondencia a outputRange y somete la salida de la función a igualar directamente el factor original evaluado I.[10]

El parámetro extra de easing altera la normalización de avance. Invoca fórmulas curvas empaquetadas en el objeto principal Easing (incluyendo algoritmos de interpolación circular o ecuaciones armónicas como Easing.bezier(x1, y1, x2, y2)) permitiendo aceleración de inercia y rebote previo al mapeo del codominio.[10]

Primitiva Dinámica de Modelado: spring()

Para simulaciones que reproducen física y tensiones orgánicas del mundo real, prescindiendo del tiempo lineal de los cuadros y calculando energía, la función spring() implementa una aproximación al oscilador armónico amortiguado. A diferencia de las transiciones Bézier, un resorte matemático depende puramente de propiedades vectoriales de energía.[13]

El comportamiento se deriva numéricamente en cada *frame* resolviendo la ecuación de segundo orden fundamental de la mecánica rotacional o traslacional:

$$m\ddot{x} + c\dot{x} + kx = F$$

Donde m corresponde a la masa inercial, c representa el amortiguamiento de rozamiento limitante de oscilación constante y k inyecta la fuerza elástica impuesta por la deformación posicional.

Topología Cruda del Parámetro config:

La instanciación en Remotion transfiere estos vectores mediante los campos inyectados en la estructura { frame, fps, config, from, to }:

Modificador de Física	Definición Analítica del Motor	Parámetro Base	Implicación Vectorial sobre la Gráfica de Onda
mass	Inercia estructural (m).	1	Alterar ascendentemente este campo multiplica la

			resistencia escalar al inicio de aceleración prolongando el desplazamiento final y ensanchando las ondas sinusoidales de rebote en los regímenes subamortiguados.[13]
damping	Coeficiente disipativo (c).	10	Induce fricción proporcional a la velocidad ($\dot{x}$). Una elevación excesiva del factor (hacia magnitudes mayores a 100) eleva la razón de amortiguamiento ζ, anulando la región compleja de la ecuación y deteniendo la oscilación, generando regresiones suaves sobreamortiguadas.[13]
stiffness	Razón de deformación (k).	100	Escala el ímpetu gravitacional hacia el centro de restauración (to). Exacerba la aceleración máxima e incrementa la velocidad inicial del frente de animación de forma lineal a su

			multiplicación.[13]
overshootClamping	Controlador estático diferencial.	false	A diferencia de los moduladores mecánicos, actúa interceptando la solución del oscilador. En el momento micro-temporal donde x cruza la barrera preestablecida to, el sistema cancela la energía residual fijando instantáneamente la velocidad terminal en cero e igualando $x =$.[13]

Restricciones de Integración y Parámetros Adicionales: La función evalúa los atributos temporales y físicos aplicando un orden estricto de prioridades matemáticas. La inserción concurrente de la configuración durationInFrames rompe la limitación infinita del resorte. Cuando se indica, la API llama al generador interno measureSpring() para identificar el punto natural de estabilización (T_{nat}) donde la oscilación residual asume una discrepancia imperceptible frente al objetivo to (establecida de manera predeterminada en un umbral de convergencia threshold = 0.005). Con esto determinado, el resorte escala la proporción del avance multiplicando el *frame* en curso por el factor $\frac{T_{nat}}{durationInFrames}$. Posteriormente, si se invoca la directriz opcional reverse: true, se efectúa la inversión calculando $T - frame$ para asegurar que la contracción del cálculo ocurra sin alteraciones de las ondas.[13]

Módulo 4: Integración de Activos Estáticos y Multimediales

El consumo de bibliotecas, gráficos de vectores y extensiones externas no deterministas dentro de un sistema basado en renderizados cerrados es intrínsecamente inestable. El procesamiento *headless* exige que la composición final congele su captura de *frame* sin ceder

control a latencias de red impredecibles.[15]

Arquitectura de Bloqueo: El Controlador delayRender y Redes Asíncronas

La sincronía garantizada depende de la invocación de delayRender(). Su instanciación emite un *handle* de retención y pausa la instrucción de compilación final de Puppeteer, forzando al evaluador a postergar la extracción matricial hasta que la promesa o el evento de red indique su finalización mediante el cierre con continueRender(handle).[16] Remotion interrumpe toda operación que exceda el marco límite natural de $30,000 \text{ ms}$, originando un fallo letal del subproceso con el volcado por tiempo agotado ("Timeout") para prevenir bloqueos silenciosos (deadlocks) del contenedor servidor.[16]

- **Implementación en <Img> y <IFrame>**: Remotion envuelve la inicialización del HTML nativo forzando pausas estáticas de re-intentos de captura de recursos para prever y asegurar el despliegue del componente. A diferencia del protocolo habitual, las etiquetas admiten la carga delayRenderRetries para establecer un cuota rígida de iteraciones tras las cuales se acciona un cierre destructivo de cancelRender() para matar operaciones sin resolución favorable. <IFrame> es instrumental porque suspende el motor maestro de Chromium en cada *frame* hasta asegurar que la ventana incrustada finalizó de calcular iteraciones topológicas y operaciones internas, logrando fusionar el reloj huésped y el alojador.[17]

Integración Geométrica Compleja: Animaciones @remotion/lottie

La inclusión de arquitecturas vectoriales complejas (JSON exportados típicamente desde Adobe After Effects o herramientas equivalentes) requiere el empaquetado del módulo cruzado lottie-web. Para eludir la discrepancia de decodificadores temporales entre el reproductor nativo de Lottie y Remotion, el envoltorio interno detiene forzosamente la propiedad autoplay.[19]

- **Manipulación Estricta de Sincronía**: Por cada invocación de actualización de componente, el método transfiere el cálculo matemático hacia Lottie utilizando el mecanismo .goToAndStop(frame). La función obliga al procesador a re-trazar los polígonos correspondientes exclusivamente a un punto estático.[19]
- **Limitaciones Deterministas Críticas**: La renderización admite únicamente el formato svg, anulando integraciones de alto rendimiento soportadas en WebGL. La inserción de JavaScript anidado o de expresiones condicionales (habituales en animaciones con interactividad como manipuladores .setSubFrame()) dentro de la estructura Lottie corrompe la extracción serial de los *frames*, precipitando que el algoritmo retorne proyecciones incoherentes o parpadeos gráficos. Las animaciones Lottie se tratan bajo la presunción de ser artefactos dependientes del tiempo, estrictamente inmutables.[19]

Operaciones con Pixeles Dinámicos y Soporte a Transparencia en Videos

La manipulación de decodificación inter-subprocesos con frecuencias discrepantes genera el riesgo de fotogramas nulos. El nodo <OffthreadVideo> neutraliza este desajuste. No genera contenedores funcionales de visualización <video> durante los entornos SSR de renderizado final, sino que deriva la extracción hacia el flujo asíncrono puro de los núcleos computacionales o binarios empotrados como FFmpeg.[20] El marco obtenido es posteriormente incrustado en el árbol estático.[22]

Evaluación del Canal Alfa: Para incrustar superposiciones complejas exentas de fondo sólido, la etiqueta admite el paso forzado paramétrico transparent={true}. La compatibilidad está matemáticamente limitada al formato WebM encapsulado bajo la matriz VP8 o VP9, aplicando muestreo de crominancia en el orden especial yuva420p.[23] La habilitación exige que el entorno subyacente soporte decodificación a través de WebGL. El motor trasvasa las texturas evaluadas a planos renderizables introduciendo, sin embargo, un detrimento logarítmico medible del rendimiento general.[22] Si los recursos son generados en h264 tradicional (careciendo de la inyección matemática alfa), la estructura puede resolverse estéticamente integrando manipuladores en línea a través de estilos condicionados de CSS puro como mixBlendMode: "screen" en el contenedor primario, lo que induce una cancelación del valor de negro subyacente.[20]

Módulo 5: CLI, SSR y Exportación Paramétrica

La consolidación y ejecución remota o sistemática de videos implica una transición radical: el código base de React (fuente) se independiza de las variables en tiempo de compilación para soportar múltiples matrices de renderizado masivo sin mutación manual. El subsistema SSR (@remotion/renderer y @remotion/bundler) aísla la carga útil.

Input Props y Abstracción Paramétrica

La parametrización se define mediante un objeto maestro estricto de formato JSON inyectado periféricamente en la compilación.[24] Estos datos transmutan los componentes gráficos de React durante la exportación final.[2]

- **Configuración del Punto de Entrada (defaultProps)**: Se establecen en el módulo <Composition>. Es de vital importancia la ausencia de serialización impura: cualquier intento de adjuntar primitivas mutables, clases heredables o variables locales ancladas al cliente fallará silenciosamente, dado que este objeto es forzado al plano serializable que Puppeteer transmitirá al contexto de renderizado del DOM en hilo separado.[6]
- **Recálculo Dinámico (calculateMetadata)**: Constituye un ciclo de interrupción condicional de la API, donde antes de iniciar el trazado inicial de píxeles, permite transformar las propiedades Input Props en metadatos re-escalados. Esto logra una

adaptación elástica y autómata de durationInFrames tras, por ejemplo, evaluar una llamada asíncrona hacia una interfaz REST que determina el valor en milisegundos reales de un recurso.[6]

- **Lectura Aislada**: El *hook* auxiliar determinista getInputProps() provee acceso inmediato a las llaves maestras en instancias del código que flotan estructuralmente por encima de la topología encapsulada del video general (para evaluaciones críticas antes de inyectar a composiciones secundarias).[25]

Arquitectura Servidor-Render (SSR: Node.js, Lambda, Bundler)

La transmutación del sistema de React a fotogramas codificados involucra llamadas secuenciales de los paquetes auxiliares @remotion/bundler y @remotion/renderer, que controlan a Node.js.[28]

Flujo SSR Básico y Local:

1. **Fase de Intercepción de Código Fuente:** La función bundle({ entryPoint, webpackOverride }) es instruida para interceptar la cabecera de la declaración general de Webpack.[28] Sintetiza toda la lógica local y resuelve un archivo maestro empaquetado retornando un *URI string* (el serveUrl). Esta salida encapsulada preserva un único estado en la compilación.[28]
2. **Validación Aislada:** La función interna selectComposition({ serveUrl, id, inputProps }) consume el paquete de la iteración previa. Efectúa una decodificación de las variables globales e implementa la topología calculada. Produce una salida en caché, indispensable para verificar que los componentes encajen antes de invocar a Chromium.[28]
3. **Invocación del Extractor:** La API maestro renderMedia({ composition, serveUrl, codec, outputLocation,... }) arranca y opera sobre el binario FFmpeg. Los metadatos evalúan cada paso del *render* (controlables mediante parámetros crudos de FFmpeg expuestos directamente a la API, tales como concurrency, directrices videoBitrate, o factores estáticos de optimización crf).[27]

Distribución a Alta Escala: Arquitectura @remotion/lambda La renderización de alta duración incurre en una sobrecarga inaceptable para arquitecturas de un solo nodo. Remotion introduce un mecanismo de Map-Reduce delegado a los centros de cálculo de AWS, abstrayendo por completo el trabajo manual de segmentación paralela.[2]

- **Pilar de Almacenamiento**: La ejecución invoca operaciones de empaquetado asíncronas mediante deploySite(), empujando los binarios minificados y los recursos de código al Bucket S3 principal de lectura rápida en regiones habilitadas (ej. us-east-1 o eu-central-1).[2]
- **Instanciación de la Capa (Layer) de Cómputo**: Remotion impone dependencias críticas sobre una función pre-estructurada y estandarizada mediante configuraciones de CLI. Un entorno Headless Chrome junto a FFmpeg residen empotrados con variables paramétricas ajustables (como limitadores de memoria y el límite temporal nativo de

AWS de 900 segundos).[2]

- **Orquestación en Cascada (renderMediaOnLambda)**: La API delega un JSON de opciones de construcción asimétrica masiva. Admite propiedades estructurales de concurrencia: la instrucción paramétrica framesPerLambda dicta la extensión métrica delegada a los fragmentos individuales.[29] El nodo maestro divide matemáticamente el segmento de durationInFrames, invoca un límite escalable forzado de hasta 200 funciones Lambda secundarias en paralelo, y espera por la decodificación individual de las sub-composiciones exportadas en fragmentos cortos.[29]
- **Ensamblaje y Finalización**: Mediante directrices asíncronas no públicas (vinculadas internamente a secuencias como combineChunks), la función AWS administradora atrae de regreso los fragmentos al directorio efímero local (/tmp, confinando el volumen terminal a 10GB brutos), concatena de manera nativa sin decodificación y re-encodificación repetida y exporta un documento crudo en la raíz externa de AWS S3 de manera atómica, finalizando en minutos secuencias que, operando en un único hilo, saturarían el motor Chromium por horas.[2]

Fuentes citadas

1. remotion-video | Skills Marketplace - LobeHub, acceso: marzo 15, 2026, https://lobehub.com/skills/mathews-tom-praxis-skills-remotion-video
2. @remotion/lambda | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/lambda
3. Remotion System Prompt for LLMs | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/ai/system-prompt
4. useCurrentFrame() | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/use-current-frame
5. The fundamentals | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/the-fundamentals
6.
7. useVideoConfig() | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/use-video-config
8.
9.
10. interpolate() | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/interpolate
11. remotion/packages/core/src/interpolate.ts at main - GitHub, acceso: marzo 15, 2026, https://github.com/remotion-dev/remotion/blob/main/packages/core/src/interpolate.ts
12. Easing | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/easing
13. spring() | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/spring
14. remotion/packages/core/src/spring/measure-spring.ts at main - GitHub, acceso:

marzo 15, 2026, https://github.com/remotion-dev/remotion/blob/main/packages/core/src/spring/measure-spring.ts
15. , and | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/use-img-and-iframe
16. delayRender() and continueRender() | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/delay-render
17. | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/img
18.
19. @remotion/lottie | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/lottie
20. Embedding transparent videos | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/videos/transparency
21.
22. Supported media | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/media/support
23. Rendering transparent videos | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/transparent-videos
24. Input Props | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/terminology/input-props
25. Passing props to a composition | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/passing-props
26. Variable duration and dimensions | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/dynamic-metadata
27. renderMedia() | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/renderer/render-media
28. Rendering using SSR APIs | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/ssr-node
29. @remotion/lambda | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/docs/lambda/api
30. Blog | Remotion | Make videos programmatically, acceso: marzo 15, 2026, https://www.remotion.dev/blog/page/2